

Toward Crypto Agility: Automated Analysis of Quantum-Vulnerable TLS via Packet Inspection

Subeen Cho¹, Yulim Hyoung¹, Hagyeong Kim¹, Minjoo Sim¹,
Anupam Chattopadhyay², Hwajeong Seo¹, and Hyunji Kim¹

¹ Hansung University, Seoul, South Korea

² Nanyang Technological University, Singapore

chosubin1208@gmail.com, Yulim4Hyoung@gmail.com, kimhaha4420@gmail.com, minjoos9797@gmail.com,
anupam@ntu.edu.sg, hwajeong84@gmail.com, khj1594012@gmail.com

Abstract. The advancement of quantum computing threatens traditional public-key cryptographic algorithms such as RSA and ECC, both vulnerable to Shor’s algorithm. As most Transport Layer Security (TLS) deployments still rely on these quantum-vulnerable algorithms for key exchange and digital signatures, the transition to Post-Quantum Cryptography (PQC), standardized by NIST, has become increasingly urgent. Given the critical role of TLS in securing Internet communications, identifying and replacing these algorithms in operational deployments is a key priority for ensuring long-term security.

We present an automated method for analyzing TLS network packets to detect the use of quantum-vulnerable algorithms. Our approach combines hierarchical packet filtering, protocol-aware parsing, and a hybrid certificate extraction technique that enables analysis of encrypted TLS 1.3 certificates without full decryption. The framework achieved over 96% detection accuracy, and our certificate parsing strategy improves overall throughput. Applying it to domestic and international TLS deployments revealed that domestic systems lag behind in quantum-readiness, underscoring the need for greater adoption of TLS 1.3, hybrid key exchanges (RSA/ECC with PQC), and short-lived certificates. Beyond TLS, the underlying methodology can be extended to other secure communication protocols, offering a versatile foundation for post-quantum migration strategies. These results highlight the practicality of our method for large-scale, real-time TLS assessments and its potential to guide PQC adoption.

Keywords: Cryptographic Agility · Transport Layer Security · Post-Quantum Cryptography · Packet Inspection · Automated Analysis.

1 Introduction

In the digital era, ensuring the security of data transmission over open networks has become a fundamental concern for individuals, enterprises, and governments alike. Transport Layer Security (TLS) protocol serves as the de facto standard for securing Internet communications, forming the backbone of online banking, e-commerce, social media, messaging, and a wide range of other services [1]. However, as cyber threats become increasingly sophisticated, defensive strategies must also evolve. In particular, the rapid advancement of quantum computing technology poses a significant threat to the security of existing modern cryptographic algorithms, especially those widely used in TLS 1.2 and TLS 1.3. Quantum computers, leveraging algorithms such as Shor’s [2], are expected to break the security of RSA [3], Diffie-Hellman (DH), and Elliptic Curve Cryptography (ECC) [4]—the core components of current TLS cipher suites—thereby fundamentally endangering the confidentiality and integrity of global digital communications.

This quantum threat has accelerated worldwide efforts to develop and standardize Post-Quantum Cryptography (PQC). Researchers and industry practitioners are actively exploring quantum-resistant algorithms in existing security frameworks to ensure long-term protection. The transition from TLS 1.2 to TLS 1.3 has brought important advances in protocol simplification, handshake efficiency, and cryptographic agility [5]; nevertheless, both versions fundamentally rely on classical public-key cryptography (i.e., RSA and ECC), leaving them inherently vulnerable in the quantum era. As a result, there is an urgent need to assess the real-world deployment of TLS cipher suites and to develop automated tools capable of detecting quantum-vulnerable configurations in operational systems.

Despite the urgency of this issue, practical real-time systems that analyze network traffic, identify cryptographic algorithms in use, and automatically detect quantum-vulnerable configurations remain scarce. This challenge is particularly pronounced in heterogeneous environments, where various software applications and operating systems may employ different TLS versions and cipher suites, often without user or administrator awareness. Furthermore, the complexity of TLS packet structures and the diversity of implementation practices pose significant obstacles to manual analysis and classification.

To address these challenges, this paper proposes a novel framework for the automated real-time detection of quantum-vulnerable cryptographic configurations in TLS traffic. The proposed system enables the collection and analysis of network packets generated by a variety of applications—including web browsers, messaging platforms, and online services—across multiple operating systems such as Windows and Kali Linux. By extracting and classifying TLS handshake information and cipher suite selections, the system can automatically determine the quantum resilience of each observed connection. Furthermore, the framework is designed to be extensible, supporting the integration of machine learning-based fingerprinting and classification to enhance the detection of previously unseen packet patterns.

1.1 Contributions

The main contributions of this study are as follows:

- **Open-source Framework for Automated Packet-Level Detection of Vulnerable TLS Configurations:** The proposed framework, released as an open-source implementation³, enables real-time analysis of deployed TLS configurations by extracting cipher suite information from network traffic. It automatically identifies insecure cipher suites and protocol implementations across diverse platforms, without requiring access to server-side keys.
- **Comprehensive Framework for Cryptographic Agility in the Quantum Era:** This work presents the framework that evaluates post-quantum readiness through automated analysis of network packet data. Designed with extensibility in mind, the framework supports not only TLS but also other network protocols, thereby enabling broader security assessments in post-quantum migration scenarios. Its effectiveness has been validated through practical experimentation across heterogeneous environments, offering both theoretical insights and operational guidance for achieving sustainable cryptographic agility and long-term system resilience in the quantum era.
- **Detection of TLS Hybrid Handshake with Emerging Post-Quantum Algorithms:** In our work, it identifies hybrid handshake packets that combine classical and post-quantum cryptography. Notably, post-quantum algorithms at NIST security level 1—offering security equivalent to AES-128—are classified as quantum-vulnerable due to their limited security margin and uncertainty under evolving quantum threat models. This provides a deliberate consideration of potential security threats against PQC in deployment scenarios.
- **Efficient and Robust Packet Filtering:** Performing exhaustive packet parsing for vulnerability analysis is inefficient and often unnecessary. To achieve both robustness and efficiency, our approach combines hierarchical filtering to isolate relevant traffic, protocol-aware parsing that leverages structural characteristics of the data. In particular, we introduce a hybrid certificate parsing method that uses lightweight OID-based matching with selective OpenSSL parsing. This enables accurate extraction of certificate fields from encrypted TLS 1.3 sessions without full decryption, supporting precise real-time analysis with reduced overhead.

2 Related Works

2.1 Transport Layer Security

Transport Layer Security (TLS) is the standard protocol for securing communications over the Internet and other open networks. Its main goals are to provide confidentiality, integrity, and authentication between communicating

³ The source code developed for this research is publicly available at <https://github.com/kpqc-cryptocraft/Crypto-Agility>. The code can be easily adapted for the analysis of other applications or environments by modifying the relevant modules.

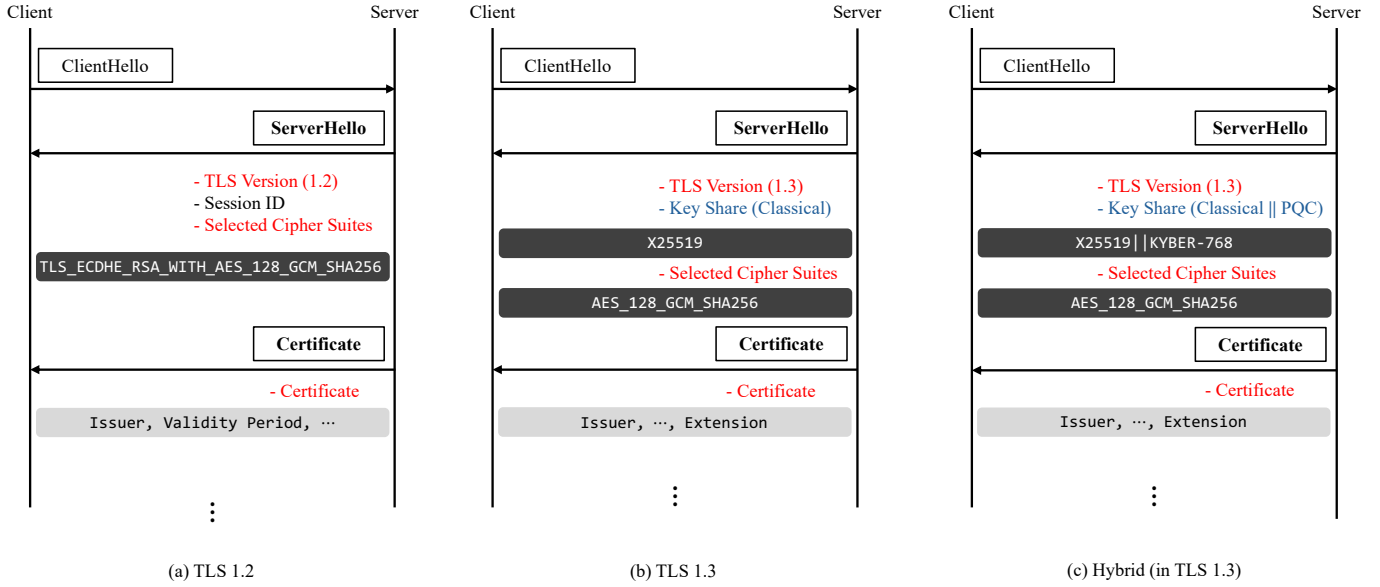


Fig. 1: Comparison of TLS 1.2, TLS 1.3, and hybrid handshake in TLS 1.3.

parties. TLS has evolved through several versions, with TLS 1.2 and TLS 1.3 being the most widely deployed today. The two versions share the same fundamental objectives but differ significantly in protocol structure, cipher suite design, handshake process, and security properties. Figure 1 illustrates the handshake sequence between the client and server in each TLS version, including key negotiation, authentication, and the establishment of encrypted communication [5].

TLS 1.2 TLS 1.2 is a widely deployed cryptographic protocol that provides confidentiality, integrity, and authentication for network communications. The protocol operates over a reliable transport layer such as TCP and consists of two main components: the Record Layer and the Handshake Layer. The handshake process in TLS 1.2 is relatively complex and involves multiple message exchanges between the client and server.

TLS 1.3 TLS 1.3, standardized in 2018, significantly simplifies and strengthens the protocol by reducing the number of handshake messages and eliminating legacy and insecure features. The handshake is designed to require only one round trip and to encrypt more of the negotiation messages, thereby improving both security and privacy. In TLS 1.3, key exchange algorithms are negotiated independently of the cipher suite through the *keyshare* extension (see Figure 1). This decoupling allows more flexible combinations of key exchange and encryption algorithms.

Hybrid Handshake in TLS 1.3 Hybrid handshake denotes a key exchange and certification mechanism wherein a classical algorithm (e.g., X25519) is combined with a post-quantum algorithm (e.g., KYBER-768) to establish a cryptographic session. This strategy is intended to provide robustness against both classical and quantum-capable adversaries. By concurrently executing both key exchange operations and deriving the session key from the aggregation of the resulting shared secrets, hybrid TLS maintains confidentiality even in the event that one of the constituent algorithms is subsequently broken. This also applies to digital signatures used for certificate, suggesting that hybrid certificates are supported (see Section 2.1).

Cipher Suites The cipher suite field represents the results of the negotiation (e.g., key exchange, authentication, encryption, and integrity protection) between the client and server of the cryptographic pa-

rameters to be used for this connection. In TLS 1.2, its format follows the structure: `TLS_key_exchange_authentication_WITH_encryption_message_authentication`.

In TLS 1.3, a cipher suite specifies only the authenticated encryption with associated data (AEAD) algorithm⁴ and the hash function for record protection, while key exchange mechanisms are negotiated separately during the handshake via distinct messages—such as `key_share` extension in the `ServerHello` message.

Certificate TLS employs X.509 certificates [6] to authenticate servers and clients. In both TLS 1.2 and TLS 1.3, the server certificate is transmitted during the handshake phase to establish trust between communicating parties. Each certificate includes critical fields such as the issuer, validity period (i.e., `notBefore` and `notAfter` timestamps), and a digital signature that ensures the authenticity and integrity of the certificate contents. Currently, RSA and ECDSA are the most widely adopted signature algorithms in deployed TLS environments.

Starting from TLS 1.3, however, the certificate message transmitted after the *ServerHello* is encrypted to prevent man-in-the-middle attacks. As a result, it becomes significantly more difficult to directly inspect certificate contents by simply capturing network traffic. This encryption poses a substantial challenge to traditional vulnerability analysis tools, which typically rely on unencrypted certificate data to assess security weaknesses. Furthermore, as previously noted, both RSA and ECDSA are known to be vulnerable to quantum attacks, highlighting the need for post-quantum secure alternatives.

Quantum-Safe Certificates To ensure long-term cryptographic resilience, a migration to post-quantum signature schemes—such as CRYSTALS-Dilithium and Falcon—is essential. A representative approach to incorporating post-quantum cryptography into certificates is the hybrid certificate, in which a classical and a post-quantum signature algorithm are used at different levels of the certificate chain. In addition to this strategy, other methods such as composite certificates [7,8], which combine multiple public keys and signatures within a single certificate structure, and parallel certificates [8], where distinct certificates are issued and validated separately for each algorithm, have also been proposed. Recent experimental studies [9,10] have explored the integration of PQC-based certificates into TLS 1.3 handshakes, providing early evidence of the feasibility of post-quantum authentication within the existing TLS infrastructure.

2.2 Quantum Vulnerable and Quantum Safe Cryptography

In 2016, the U.S. National Institute of Standards and Technology (NIST) highlighted, based on expert consensus, the increasing likelihood that the RSA algorithm would become vulnerable by the year 2030 due to advancements in quantum computing [11]. To address the potential cryptanalytic threat posed by Shor’s algorithm—which can efficiently break widely used public-key cryptosystems such as RSA and elliptic curve cryptography (ECC)—NIST launched the Post-Quantum Cryptography (PQC) standardization process at the end of 2017 [2].

A total of 69 candidate algorithms were submitted in the first round, and a multi-year evaluation was conducted based on criteria such as security against quantum and classical attacks, implementation efficiency, and algorithmic robustness. These developments highlight the pressing need to transition from legacy cryptographic systems, such as RSA and ECC, to quantum-resistant alternatives. This effort culminated in August 2024 with the finalization of three new standards:

- **FIPS 203:** Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM)
- **FIPS 204:** Module-Lattice-Based Digital Signature Standard (ML-DSA)
- **FIPS 205:** Stateless Hash-Based Digital Signature Standard (SLH-DSA)

In addition to these, the FALCON signature scheme is currently under consideration for standardization [12], and HQC has been newly selected as an additional KEM algorithm [13]⁵.

⁴ AEAD algorithms provide both confidentiality and integrity by combining encryption and authentication in a single cryptographic primitive.

⁵ These PQC schemes are available via the OQS library, an open-source project aimed at integrating quantum-resistant cryptographic algorithms (see <https://github.com/open-quantum-safe/liboqs>.)

These standardized algorithms are parameterized by NIST-defined post-quantum security levels (see Table 1 in [14]), ranging from Level 1 to Level 5. Notably, Security Level 1 corresponds to a security strength comparable to AES-128, and is often used as a baseline for evaluating the quantum resistance of proposed schemes.

Similar to public-key cryptography under Shor’s algorithm, symmetric-key cryptography may also face potential quantum vulnerabilities. Grover’s algorithm [15] provides a quadratic speedup by reducing the complexity of searching an unsorted space of size N from the classical $O(N)$ to the quantum $O(\sqrt{N})$, posing a threat to symmetric-key cryptography (though less critical than Shor’s algorithm). Symmetric-key algorithms such as AES-128 and SHA-256 are not completely broken by quantum algorithms, but doubling the key size is generally recommended to achieve quantum-safe security level.

To achieve this transition effectively, it is crucial to first analyze the current deployment of cryptographic algorithms across diverse systems and environments. Such an understanding provides the foundation for secure migration strategies. In this paper, we focus on the detection of cryptographic vulnerabilities, particularly the identification of insecure or quantum-vulnerable cryptographic primitives that are still in use, as a foundational step toward a comprehensive PQC transition.

2.3 Security Vulnerabilities in TLS

The security properties and vulnerabilities of TLS primarily stem from its protocol version and supported cipher suites. Accordingly, we categorize them into protocol-level vulnerabilities and quantum vulnerabilities (QV) based on the applicable attack (see Table 1).

Table 1: Analysis of vulnerability in TLS 1.2 and TLS 1.3.

TLS Version	Function	Cryptographic algorithm*	Vulnerabilities
TLS 1.2	Key Exchange	RSA, ECDHE, ECDSA	S, D
	Encryption	RC4, 3DES, AES-128	G, D
	Hash	SHA-256	G, D
TLS 1.3	Encryption	AES-128	G
	Hash	SHA-256	G

S, G, and D: Shor’s algorithm, Grover’s algorithm, and downgrade attack.

*: Only quantum vulnerable or deprecated algorithms are listed.

Protocol Vulnerability TLS 1.2 greatly improved security over previous versions, but remains vulnerable if legacy options are enabled. Notable risks include lack of mandatory forward secrecy, support for deprecated algorithms (e.g., RC4, 3DES), and susceptibility to attacks such as POODLE and Lucky13 [16,17]. TLS 1.3 addresses these issues by enforcing forward secrecy, removing insecure algorithms, encrypting more handshake data, and providing built-in downgrade protection.

Quantum Vulnerability Even in the case of TLS 1.3, it is difficult to ensure security in a quantum computer environment because the handshake process is based on public-key cryptographic algorithms that are vulnerable to Shor’s algorithm. Moreover, TLS employs symmetric-key cryptography for both encryption and hashing, which can be weakened by Grover’s algorithm due to its quadratic speedup of exhaustive search. On a related note, although is generally considered secure, recent research on various attacks against PQC [18,19,20] suggests that continued attention of its security strength remains essential. For example, SIKE was broken on a CPU for NIST security level 1 parameters [21]. Collectively, these developments highlight the necessity of continuous reassessment of cryptographic schemes previously regarded as robust in a post-quantum security.

Accordingly, quantum vulnerabilities in PQC must be taken into account. In this work, our analysis includes PQC algorithms that offer a security level comparable to AES-128, as defined by NIST.

2.4 Cryptographic Agility

Cryptographic agility refers to the capability of a system to transition between cryptographic algorithms or configurations with minimal operational disruption. As highlighted in NIST’s Cyber Security White Paper (CSWP) [22], crypto agility is not merely the ability to support multiple algorithms, but rather hinges on the ability to accurately identify which algorithms, cipher suites, and protocol parameters are currently in use. This identification enables timely responses to newly discovered vulnerabilities, algorithm deprecation, or evolving compliance requirements.

Cryptographic Agility for TLS Cryptographic algorithms are used to deliver essential security properties, including confidentiality, integrity, authentication, and non-repudiation. These protocols require communicating parties to negotiate a common set of algorithms—referred to as a cipher suite—through which such properties are realized. In this context, the capability to explicitly identify the cryptographic algorithms in TLS is essential. Accordingly, this work aims to identify protocol configurations and cryptographic algorithms that potentially undermine the security of TLS, thereby establishing the anticipatory response capability required for the implementation of cryptographic agility.

2.5 Overview of TLS Packet Analysis Technologies

In modern network environments, the TLS protocol is extensively utilized across a broad spectrum of applications, including web browsers, messenger services, and cloud platforms. However, each application employs a distinct TLS implementation, resulting in differences in supported cipher suites and security policies, which clearly limits the effectiveness of manual analysis. For example, Chrome and Firefox adopt different cipher suite priorities depending on their respective versions and configuration settings. Consequently, the development and deployment of reliable packet analysis tools and automated frameworks have become essential requirements.

Focus and Gaps in Previous Works Previous works have primarily focused on structural analysis of the TLS protocol, surveys of cipher suite distributions, or server-side vulnerability detection. However, most of these studies has concentrated on traffic classification or malicious traffic detection, and there is a notable lack of work that detects quantum-vulnerable cryptographic algorithms in real-time within actual network environments and automatically classifies the results. In particular, automated systems that systematically evaluate the quantum vulnerability of cryptographic components used in TLS packets remain scarce.

In contrast, the framework proposed in this paper offers substantial contributions by addressing these limitations of existing research. The proposed framework automates all stages, including packet collection, protocol parsing, and cipher suite extraction. By combining real-time packet capture with a PQC standards-based parsing engine, it systematically evaluates the quantum vulnerability of cryptographic algorithms and key exchange mechanisms used in network communications. Based on these observations, the following section provides a comparison of existing tools and platforms related to TLS cryptographic analysis.

Comparison with Existing Tools Table 2 presents a comparative analysis of existing studies, focusing on automated systems that systematically evaluate the quantum vulnerability of cryptographic components used in TLS packet communication. A ✓ indicates that the tool supports detection of quantum vulnerabilities in TLS traffic. A ✗ symbol denotes the absence of support for detecting cryptographic algorithms that are vulnerable to quantum attacks. Unlike prior works, the proposed framework specifically targets hybrid TLS configurations and provides automated detection of quantum-vulnerable components. Notably, it extends its detection scope to include PQC algorithms corresponding to NIST-defined security level 1, in addition to legacy cryptographic algorithms.

IBM Guardium [23] is an enterprise data security platform that provides TLS traffic inspection and risk assessment, including the detection of cryptographic configurations potentially vulnerable to quantum threats. However, it does not explicitly support automated detection or classification of hybrid key exchange configurations in the context of PQC standards. TYCHON [24] is a cybersecurity platform offering endpoint visibility and behavioral analytics, with TLS metadata analysis to identify insecure or non-compliant cryptographic configurations. A-Packets [25] is a TLS traffic analysis framework focused on detecting deprecated cipher suites such as RC4 and 3DES. Unlike

Table 2: Comparison of related works for automated analysis of TLS packet.

Reference	TLS Type	QV Detection	Target algorithms
IBM Guardium [23]	Classical	✓	Legacy ciphers
TYCHON [24]	Classical	✓	Legacy ciphers
A-Packets [25]	Classical	✗	Deprecated ciphers
This Work	Classical, Hybrid	✓	Legacy ciphers, PQC

these works, our approach supports hybrid TLS deployments and enables real-time identification of both legacy and PQC algorithms.

Table 3: Overview of enterprise-grade commercial tools supporting PQC migration.

Product	Scope	Supported Protocols
Palo Alto [26]	Firewall & governance	TLS 1.3
PQStation [27]	Cryptographic management	TLS, SSH, VPN
SandboxAQ [28]	Governance	TLS, File systems

In addition to the TLS-focused tools summarized in Table 2, several broader commercial tools address cryptographic visibility and PQC migration at an enterprise scale (see Table 3). These offerings extend beyond TLS inspection to support multiple protocols—such as SSH, VPN, and application-level encryption—thereby providing end-to-end coverage of cryptographic assets. A unifying characteristic is their orientation toward post-quantum migration strategies, which combine cryptographic inventory, risk assessment, and phased deployment of quantum-resistant algorithms. By targeting diverse protocols and enterprise-wide infrastructures, these commercial solutions highlight the growing industry focus on comprehensive readiness for the post-quantum era [26,27,28].

While commercial portfolios excel in delivering enterprise-grade governance and migration capabilities at scale, we propose an open-source framework that aims to provide transparency and accessibility, offering a research-oriented resource for further study and experimentation.

3 Proposed Methods

3.1 System Overview and Objectives

The proposed method achieves quantum-safe TLS security by automatically detecting and evaluating the vulnerability of TLS cryptographic elements from network packets. It comprises three core components: packet filtering, vulnerability detection, and evaluation with result report (see Figure 2).

Our system captures network packet data from the TLS handshake process during client-server communication and automatically analyzes the packets to identify potential quantum vulnerabilities. The evaluation focuses on three key elements: TLS version, cipher suite, and the certificate. Each element is assessed based on its security level—classified as high, medium, or low. The results are presented in a compact format: `TLS_Evaluation result_CS_Evaluation result_CA_Evaluation result`. For instance, if TLS 1.3 is used with AES-128 and SHA-256, the result is represented as `TLS_H_CS_M`. The certificate is also evaluated based on its expiration date: if it is set to expire within 90 days⁶, it is considered due for renewal. Therefore, Certificates with an expiry date within 90 days are marked as `True`, while all others are marked as `False`.

⁶ Certificates are generally renewed within 90 days prior to their expiration. See DigiCert documentation <https://docs.digicert.com/en/certcentral/manage-certificates/renew-an-ssl-tls-certificate.html>.

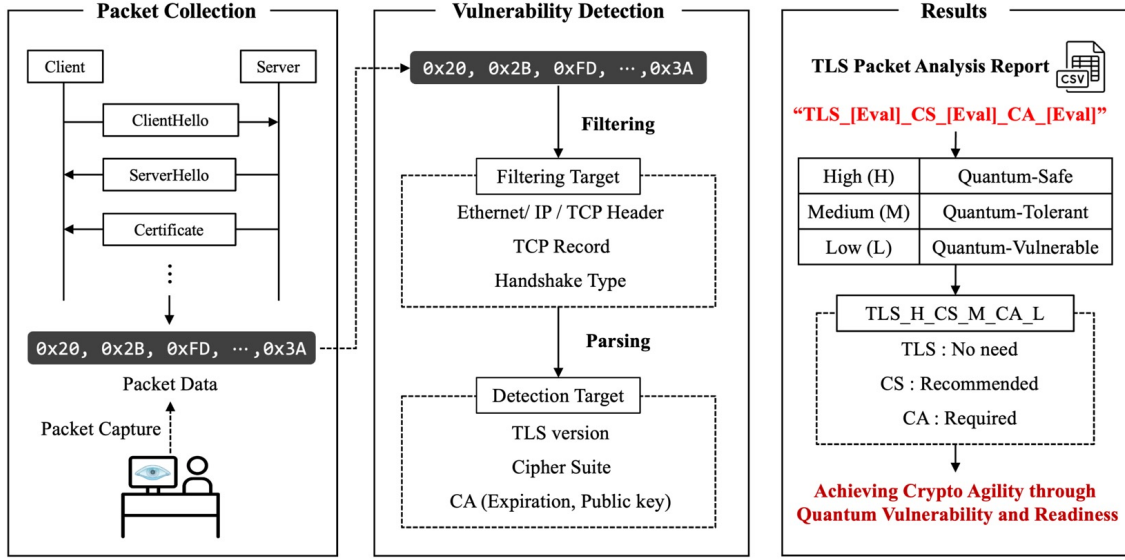


Fig. 2: System Overview.

In our scenario, we also account for scenarios in which packets are not properly transmitted. A TLS Record must inherently include a fixed-size Record Header of 5 bytes and at least 1 byte for the Handshake Type. Therefore, its total length must be at least 6 bytes to include the handshake type and a minimal message body [1,5]. If this condition is not satisfied, the packet is considered incomplete, and the analysis is terminated. Moreover, the parsing process begins by interpreting the length fields preceding each data field, followed by the analysis of the corresponding field values. If a mismatch or truncation in the specified field length is detected, parsing of the affected field is immediately terminated to prevent erroneous interpretation.

3.2 Efficient Packet Filtering

Prior to the detection phase, we capture network traffic to obtain raw packet data. Since the packets vary in type, we apply a two-phase filtering process to extract only the traffic relevant to our analysis. To improve the reliability of our work, we collected most packets on the selected network interface in promiscuous mode⁷, while minimizing packet loss and truncation.

Network Filtering To improve analytical efficiency and avoid unnecessary inspection of the entire network, the proposed system employs a traffic targeting mechanism. It supports two types of targeting: selection of the network interface and Server Name Indication (SNI) filtering. Packets are first filtered from a user-specified network interface, after which only traffic associated with selected domains is retained for analysis. This hierarchical filtering approach significantly reduces the analysis scope, enabling focused inspection of relevant traffic and facilitating efficient packet analysis.

Packet Filtering Figure 3 illustrates the structure of the Ethernet, IPv4, and TCP headers. Each of these headers contains multiple fields, and they are filtered in sequence to identify and extract TLS handshake packets. The filtering process yields the TCP payload, which is then subject to vulnerability analysis. The purpose of this procedure is to precisely determine the offset of each protocol layer (i.e., Ethernet, IP, and TCP layers) to reduce parsing error. To achieve this, field-level information is extracted based on standardized header structures.

⁷ Promiscuous mode allows the network interface to receive all packets on the network segment, regardless of their destination address.

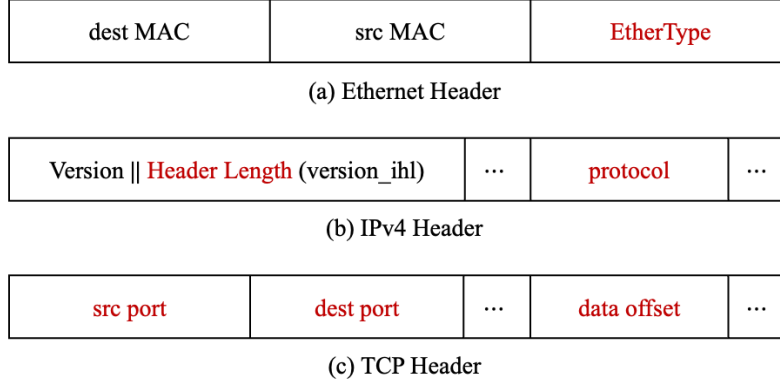


Fig. 3: Header Structure of Ethernet, IPv4 and TCP.

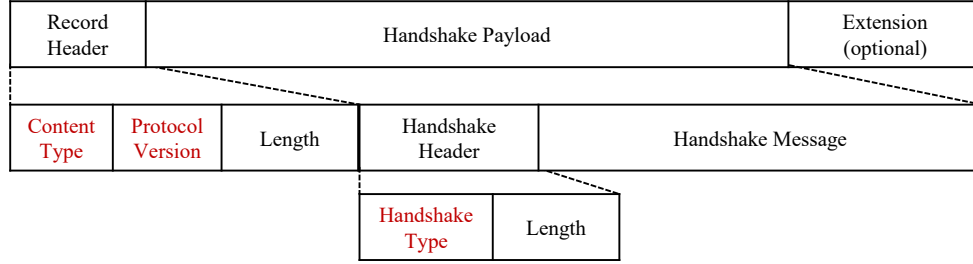


Fig. 4: TLS Record structure for handshake type.

Ethernet Header Filtering The Ethernet header has a fixed size of 14 bytes, and packets are passed to the next parsing step only if the ethernet type field indicates IPv4 (i.e., it has a field value of 0x0800) [29]. Since the vast majority of real-world Internet traffic—particularly in the context of encrypted communication and attack detection—is based on IPv4, non-IPv4 packets are excluded from subsequent processing⁸. Although IPv6 is the next-generation Internet protocol, many mainstream service environments still rely heavily on IPv4; thus, IPv6 traffic is excluded due to its limited real-world analytic relevance [30].

IPv4 Header Filtering The IPv4 header is typically 20 bytes when no options are present, but its length varies if options are included. The *version_ihl* field in the IPv4 header includes both the version and header length fields, and extracting the actual header length enables accurate processing of packets with variable-length headers. In this filtering process, only packets with the *protocol* field set to TCP (i.e., a field value of 0x06) are retained for subsequent processing. This is because virtually all encrypted application-layer traffic (e.g., HTTPS) relies on TCP, and excluding non-TCP packets reduces unnecessary overhead in the analysis [31].

TCP Header Filtering Similar to the IPv4 header, the TCP header parsing step requires extracting the header length. This is accomplished by interpreting the *data_offset* field, which specifies the length of the TCP header.

TLS Record Filtering A TLS Record is a logical data transmission unit used in the TLS protocol to convey actual application data and control messages. Each TLS Record consists of a record header and a payload [32]. The location of the TLS Record can be determined by adding the previously computed length of the TCP header to the starting position of the TCP layer. This is because the TLS Record section begins immediately after the TCP header ends. Furthermore, the length of the TLS Record is calculated by subtracting the precomputed lengths of the Ethernet, IP, and TCP headers from the total packet length.

⁸ See <https://www.potaroo.net/ispcol/2025-01/bgp2024.html>.

TLS Handshake Type Filtering TLS records include essential information required for handshake-type filtering, including the content type, protocol version and handshake header fields (see Figure 4). The presence of a handshake message is determined by the value of the content type field. If the TLS record corresponds to a handshake message, it contains a handshake header, wherein the first field specifies the type of handshake message.

As cryptographic vulnerability analysis primarily depends on key exchange and authentication information, only three handshake types are parsed in this work: *ClientHello*, *ServerHello*, and *Certificate* types, which correspond to field values 0x01, 0x02, and 0x0B, respectively. They are the parts where cryptographic negotiation information and certificates are first explicitly specified.

Port Filtering In the subsequent step, TLS-relevant traffic is extracted, as it is selectively retained through filtering based on destination port numbers. Therefore, to enable efficient traffic analysis, the system provides the ability to select specific port numbers, thereby narrowing the target traffic scope and reducing unnecessary processing. Specifically, this work focuses on traffic associated with ports 443, 8443, and 4450. Port 443 is the default for HTTPS and serves as the global standard for TLS-encrypted communications [33], while port 8443 is commonly used as an alternative SSL/TLS service port in major web application servers (e.g., Tomcat and Jetty) [34,35]. Port 4450 is additionally included based on user-specific configurations.

These port selections are intended to reflect the primary operational and attack paths in real-world Internet and enterprise network environments, which rely on encrypted traffic. In addition, we provide analysis of practical security vulnerabilities associated with standard-based services by using the chosen port number.

3.3 Vulnerability Detection

In this section, we describe the parsing procedure used to detect cryptographic elements relevant to vulnerability analysis. Our work aims to extract essential cryptographic information from TLS packets. To this end, we focus on handshake-type records, which are filtered in the previous stage (see Section 2.1). From these filtered records, the system selectively parses only the fields that are directly relevant to the vulnerability assessment.

Detection Targets Handshake-type records contain several protocol fields (e.g., server version, session ID and extension⁹). Each of these fields may encapsulate multiple subcomponents that include security-relevant information. Table 4 summarizes the selected TLS handshake fields and their corresponding extracted elements used for cryptographic vulnerability evaluation.

Table 4: Selected fields and information of TLS handshake.

Handshake Type	Field	Extracted Information
ClientHello	Extension	SNI
ServerHello	Cipher Suite	cryptographic algorithm
	Extension	protocol version, key share
Certificate	Certificate	public key algorithm, issuer, validity expiration

SNI It is used for domain filtering and session identification. Within the SNI field, the type of SNI is distinguished by the name type; however, in practice, only the domain type (i.e., a type value of 0x00) is utilized [37]. As a result, the parsed SNI value is limited to domains, allowing domain information to be directly extracted without requiring further differentiation. This enables selective traffic analysis based on user-configured domain addresses, which is essential for targeting specific TLS sessions during vulnerability evaluation.

⁹ In TLS 1.2, extensions are optional; however, in TLS 1.3, certain extensions are mandatory, resulting in a consistently present extension structure [36].

Cipher Suite The cipher suite field, which specifies the collection of algorithms used for encryption, key exchange, and authentication, serves as a critical indicator for assessing quantum vulnerability and overall security. In this work, to comprehensively analyze the latest standards in network security and recent trends in post-quantum cryptography adoption, the focus is placed on the representative protocols of TLS 1.2 and TLS 1.3 (conventional and hybrid handshake).

In the ServerHello structure, the length from the Handshake Type field to the Random field is always 44 bytes, while the length of the Session ID varies. Therefore, the starting offset of the cipher suite is determined by adding the Session ID length to 44 bytes.

Parsing Strategies for Robust and Efficient Detection To ensure accurate and reliable vulnerability detection, we propose two core parsing strategies. These methods are applied in the certificate detection phase and are designed to address structural variability and practical limitations associated with real-world TLS packet inspection.

Protocol Version This field (i.e., *supported_versions*) in the TLS record header is used to indicate the TLS version employed in a given session. Legacy TLS versions ($\leq$ TLS 1.1) are officially classified as historic by [38]. Fallback to these versions is strictly prohibited, and they are regarded as insecure protocols. For backward compatibility, TLS 1.3 mandates that the protocol version field be set to the same value as TLS 1.2 (i.e., its field value is 0x0303) [1]. Consequently, it is not possible to distinguish between TLS 1.2 and TLS 1.3 solely based on the protocol version field. Instead, the distinction is made by inspecting the presence of the *supported_versions* extension, which explicitly indicates support for TLS 1.3 and enables proper version negotiation between peers.

Key Share In TLS 1.3, cipher suites are composed solely of AEAD and hash algorithms [1]. Consequently, the *key_share* extension is mandatory in TLS 1.3 to facilitate key exchange. Therefore, key exchange information cannot be determined from the cipher suite alone, necessitating additional parsing. The key share algorithm value is located 4 bytes from the starting offset of the *key_share* extension.

Public Key Algorithm The public key algorithm is identified based on the OID value specified in the X.509 certificate [39]. Public Key Algorithm is parsed for comparison with the cipher suite, parsing is performed. In principle, both should match; however, inconsistencies may arise due to system errors or certificate forgery, such as man-in-the-middle attacks [40]. This comparison enables detection of potential tampering by verifying the consistency between the certificate and the handshake negotiation.

As target algorithms, we consider RSA and ECDSA, which are widely recommended for use in TLS 1.2 and TLS 1.3 [41]; Ed25519, standardized in TLS 1.3 [1]; and ML-DSA 44, a post-quantum digital signature algorithm recently standardized by NIST [42].

Issuer Issuer field is parsed to evaluate the trustworthiness of the certificate issuer. Although the issuer field in a certificate contains various subfields as defined in the standard [43], the Common Name (CN) is considered the most suitable identifier for general users to assess trustworthiness. Therefore, the CN field¹⁰ is chosen for parsing and output as the issuer's name.

Validity Expiration The validity expiration date of a certificate is a primary criterion for assessing the need for renewal, as it defines the end of the certificate's validity period. Therefore, it is essential for providing guidance on appropriate migration timelines. The ASN.1 TIME field is encoded in either UTCTime or GeneralizedTime, each adopting a unique format and length. Due to these variations, it is difficult to parse the field accurately using simple pattern matching techniques [44]. For this reason, direct parsing based on byte matching is not employed; instead, parsing is conducted using the OpenSSL library. By using the OpenSSL, the entire certificate is transformed into an X.509 structure, which grants direct access to the necessary fields. Consequently, there is no need to manually identify the position of the validity period, as a pointer to the *notAfter* field can be instantly retrieved from the X.509 structure to access its value.

¹⁰ To obtain the issuer's CN, the process involves searching the certificate for the CN OID byte value. The OID value is \x06\x03\x55\x04\x03.

Effective Certificates Detection While the parsing procedure for certificate fields is consistent across TLS versions, the method of certificate acquisition differs. Conventional TLS packet analysis tools primarily evaluate vulnerabilities based on certificate information contained in captured traffic. However, since the certificate message is encrypted in TLS 1.3 and beyond, extraction of certificates by packet capture alone is impossible (see Section 2.1). Therefore, additional decryption procedures are required, such as obtaining secret keys or session keys (e.g., session key log file) from the client side to verify certificate information [45,46].

To reduce this overhead, we propose an effective certificate acquisition method tailored for TLS 1.3. Our system introduces a dynamic server connection method utilizing the SNI value. The system immediately initiates a separate TLS client connection using the extracted SNI information via the OpenSSL library, and performs a full handshake to retrieve the certificate chain. This approach fundamentally eliminates the overhead associated with decrypting encrypted certificate messages, and significantly enhances the completeness and reliability of vulnerability analysis by acquiring up-to-date certificate data in real time.

Parsing all packets using an OpenSSL-based approach is not chosen due to the significantly higher efficiency of simple byte matching in terms of processing time. In practice, performing full certificate parsing with OpenSSL requires considerable additional time compared to simple byte matching¹¹.

As the goal of this work is real-time packet analysis, it is necessary to minimize processing delays. Therefore, a byte-matching method is prioritized in cases where information can be extracted without decryption. Only when decryption is not possible or packet errors prevented information retrieval is OpenSSL-based matching employed.

Precise OID-based Extraction of Certificate Fields The public key algorithm and issuer fields are extracted from the certificate by searching for their respective OID-encoded values across the entire byte sequence. For example, there is a certificate represented as a 32-byte hexadecimal sequence. Within this data, only the public key algorithm and issuer fields appear in the form of OIDs, making them uniquely identifiable. The OID sequences used for these fields are typically between 9 and 12 bytes in length, and are unlikely to be confused with other content. In particular, both the public key algorithm and issuer are independently encoded, each occupying at least 5 bytes, which further reduces the likelihood of structural collisions elsewhere in the certificate. This structure-agnostic approach was adopted because the standard X.509 certificate format allows variable field lengths and non-fixed ordering, rendering offset-based parsing unreliable (see Section 2.1). As a result, parsing based on full-field OID values provides a reliable method for identifying these fields even in variable-length certificate data.

Table 5: Comparison of certificate parsing strategies.

Component	OpenSSL-Only Parsing	This work
Public key	OpenSSL	OID
Issuer	OpenSSL	CN/ORG OID
X.509 conversion	Always performed	On-demand
Validity period	OpenSSL	
Execution time (ms/certificate)	182.5720	179.1399

Advantages of the Certificate Parsing Strategy Table 5 summarizes the operational differences between the conventional OpenSSL-based parsing method and our proposed hybrid approach. In the traditional model, all certificate-related fields—such as the public key algorithm, issuer, validity period, and X.509 conversion—are invariably processed through the OpenSSL API. While this ensures structural correctness, it incurs unnecessary computational overhead, particularly for real-time analysis scenarios.

¹¹ Performance concerns with the OpenSSL `d2i_X509` function for ASN.1 DER-encoded certificate parsing have been previously discussed. <https://github.com/openssl/openssl/issues/17950>.

In contrast, our hybrid method employs selective OID-based pattern matching for public key and issuer extraction, invoking OpenSSL parsing only when pattern matching fails. The same principle applies to issuer identification, where CN/ORG OID searches are prioritized and OpenSSL is used as a fallback. The validity period remains consistently parsed using the OpenSSL API due to the ASN.1 TIME structure, ensuring accuracy. X.509-to-structure conversion is deferred until explicitly required, further reducing redundant processing.

Most importantly, this hybrid strategy achieves a substantial speed advantage: when OID matching succeeds, only lightweight byte-pattern operations are performed, eliminating the latency of full OpenSSL parsing. Based on ten test runs over 597 certificates, the proposed hybrid approach achieved an average processing time of 179.1399 ms per certificate, compared to 182.5720 ms for the conventional method, representing a reduction of 3.4321 ms per certificate on average. While this corresponds to a relative improvement of 1.88%, the effect can become substantial when accumulated over an entire day of large-scale packet analysis under similar workload conditions.

3.4 Vulnerability Evaluation

Finally, the vulnerability of TLS packets is evaluated based on the security-relevant information extracted from the filtered packets. Each level reflects both classical and quantum-era considerations, incorporating protocol versions, algorithm strength, and susceptibility to known attacks. To adopt a conservative stance toward security threats, the cryptographic level is determined based on the weakest algorithm among the key exchange, encryption, and hash functions used in the cipher suite.

- **High (H):** TLS 1.3 is classified as high level due to its enhanced protocol-level security features, such as removal of insecure cipher suites, and encrypted handshake metadata. Additionally, symmetric primitives and post-quantum algorithms conforming to NIST post-quantum security levels 2–5 (e.g., AES-256, ML-DSA, ML-KEM-1024) are considered quantum-resistant even under quantum adversaries equipped with Grover’s algorithm. This level represents strong cryptographic assurance under the quantum threat models.
- **Medium (M):** This category includes TLS 1.2 and symmetric primitives such as AES-128 and SHA-256. Although TLS 1.2 remains widely deployed, its support for legacy components (e.g., CBC mode, RSA) may expose it to downgrade attacks or implementation flaws. This level reflects moderate quantum resistance, with classical security largely intact but quantum resilience limited.
- **Low (L):** This includes TLS versions (≤ 1.1) and quantum-vulnerable public-key systems, which are considered insecure due to both cryptographic and protocol-level flaws, regardless of quantum considerations. They fail to meet modern security expectations and should be phased out in any security-sensitive environment.
- **True and False (T/F):** This criterion applies only to certificates to note the migration timing: certificates set to expire within 90 days are marked as True, while all others are marked as False (see Section 3.1).

Figure 5 presents an example of the analysis output generated by our system, illustrating the results of end-to-end TLS traffic evaluation. Our work provides the foundational stage of cryptographic agility by conducting a comprehensive and fully automated analysis of TLS vulnerabilities. The system covers the entire workflow—from packet collection to evaluation and structured reporting—ensuring that potential weaknesses in TLS communications are systematically captured without manual intervention. This contribution offers practical guidance for establishing a transition roadmap at this critical juncture before the advent of the post-quantum era.

4 Results

In this section, we present the results and discussion of our proposed approach. In particular, we provide an in-depth analysis of quantum vulnerability and quantum readiness, along with a case study focusing on domestic (i.e., South Korea) and global enterprises.

4.1 Experimental Setup

Table 6 summarizes the experimental setup including operating systems, packet capture tools, and PQC libraries used in this work. In this work, we collected and analyzed a large volume of real-time TLS packets generated by 20

SNI	TLS		Cryptographic Algorithm			Certificate				Total Result
	Ver.	Lv.	Cipher Suite	Key Exchange	Lv.	Issuer	Public Key	Expiry Date	Migration	
<u>Apple</u>	1.3	H	AES_128_GCM_SHA256	x25519	M	Apple Public EV Server ECC	ECDSA	2025-09-17	T	TLS_H_CS_M_CA_L
<u>Chrome</u>	1.3	H	AES_128_GCM_SHA256	mlkem768_x25519	L	WE2	ECDSA	2025-09-29	T	TLS_H_CS_L_CA_L
<u>Naver</u>	1.3	H	AES_128_GCM_SHA256	x25519	M	DigiCert TLS Hybrid ECC SHA384	ECDSA	2026-03-17	F	TLS_H_CS_M_CA_L
<u>Google</u>	1.3	H	AES_128_GCM_SHA256	mlkem768_x25519	M	WE2	ECDSA	2025-09-29	T	TLS_H_CS_M_CA_L
<u>YES24</u>	1.2	M	AES_128_GCM_SHA256	ECDSA	L	Thawte RSA	RSA	2025-09-07	T	TLS_M_CS_L_CA_L
<u>Samsung</u>	1.3	H	AES_256_GCM_SHA384	x25519	H	GeoTrust TLS RSA	RSA	2026-02-13	F	TLS_H_CS_H_CA_L
<u>YouTube</u>	1.3	H	AES_128_GCM_SHA256	mlkem768_x25519	M	WE2	ECDSA	2025-09-29	T	TLS_H_CS_M_CA_L
<u>YES24</u>	1.2	M	AES_128_GCM_SHA256	ECDSA	L	Thawte TLS RSA	RSA	2025-09-07	T	TLS_M_CS_L_CA_L
<u>Netflix</u>	1.3	H	AES_128_GCM_SHA256	x25519	M	DigiCert Secure Site ECC	ECDSA	2025-09-24	T	TLS_H_CS_M_CA_L
<u>PayPal</u>	1.3	H	AES_128_GCM_SHA256	mlkem768_x25519	M	DigiCert EV RSA	RSA	2026-05-12	F	TLS_H_CS_M_CA_L

Fig. 5: Example of our final evaluation report.

major enterprises (e.g., Google, YouTube, and KakaoTalk) across both Windows 11 and Kali Linux environments. This dataset enabled a comprehensive evaluation of the proposed framework’s effectiveness across heterogeneous platforms and real-world usage scenarios.

Table 6: Experimental environment including operating systems and libraries.

OS	Library		
	Packet capture	PQC	Crypto/Comm.
Windows	WinPcap	OQS	OpenSSL
Linux	libpcap		

In addition, we use WinPcap on Windows and libpcap on Linux to capture packets, both of which provide access to raw packet data including the captured length. For post-quantum cryptography, we use the Open Quantum Safe (OQS) library. Its core component, liboqs, provides a unified API for post-quantum key encapsulation and digital signature algorithms across platforms. We integrate liboqs with OpenSSL via the oqs-provider plugin, enabling PQC-based TLS handshakes for benchmarking and interoperability testing. Lastly, OpenSSL is used to perform all cryptographic operations and TLS communications throughout the experiments.

4.2 Performance

Confusion Matrix We utilize a multi-class confusion matrix (with classes High, Medium, and Low) to visualize the classification results of our system across the three key TLS-related fields. Each matrix compares the predicted labels against manually verified ground truth, enabling observation of the distribution of true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN) for each class. We collected and classified

quantum vulnerability and readiness data from a total of 192 TLS sessions (SNI) through real-time packet analysis. This confirms that the three core elements—TLS version, cipher suite, and PQC migration requirements—were successfully classified.

Figure 6 visualizes the confusion matrix for each classification item. For example, in the case of (a) TLS version classification, there were 164 actual TLS 1.3 (High) sessions out of 192 total, and our system correctly classified 158 of these as High. Similarly, out of 28 actual TLS 1.2 (Medium) sessions, 27 were correctly classified as Medium. This demonstrates that the proposed system achieves a very high degree of accuracy across the TLS version, cipher suite, and migration requirement categories.

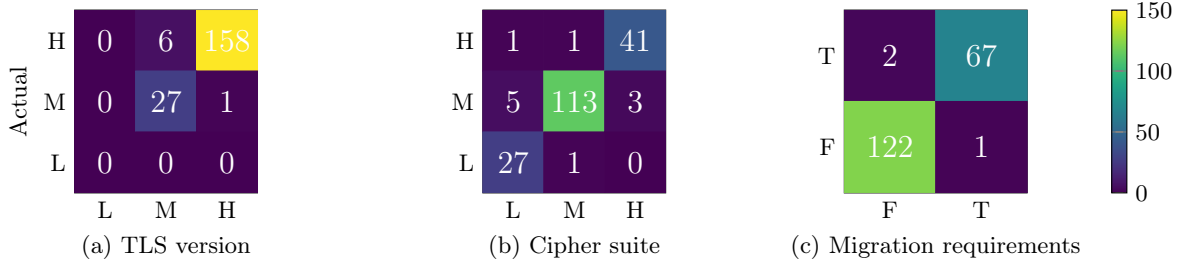


Fig. 6: Visualization of the confusion matrix of TLS version, cipher suite, migration requirements for our results.

Recall To evaluate the performance of our system, we focus on *recall*¹², which quantifies the proportion of actual positives correctly identified by the system. Recall is formally defined as:

$$recall = \frac{TP}{TP + FN}$$

A high recall indicates that the system successfully captures the majority of relevant instances, which is particularly critical in security contexts to minimize overlooked vulnerabilities. In our evaluation, recall is computed separately for each key element—TLS version, cipher suite, and certificate—to assess how accurately the system extracts each field from TLS packet data.

Table 7 shows the recall results of our system. The overall recall for TLS version classification is approximately 96.35% (= 185/192), with the TLS 1.3 class achieving 96.34% (= 158/164) and the TLS 1.2 class achieving 96.43% (= 27/28). For cipher suite classification, the overall recall is approximately 94.27% (= 181/192), with the High class achieving 95.35% (= 41/43), the Medium class achieving 93.39% (= 113/121), and the Low class achieving 96.43% (= 27/28). For PQC migration requirement classification, the overall recall is approximately 98.44% (= 189/192), with the True class achieving 97.10% (= 67/69) and the False class achieving 99.19% (= 122/123). These results indicate that the proposed system can accurately extract critical security information from real-time TLS packets, ensuring that potential vulnerabilities are not overlooked.

4.3 Application to Real-World Enterprise TLS Traffic

Quantum-readiness is comparatively analyzed for both domestic and global enterprises based on the vulnerability evaluation proposed in this work. To provide a structured discussion, we first clarify the scope of enterprise evaluation before presenting detailed results.

Scope of Enterprise Evaluation The domestic enterprises evaluated include Naver, Coupang, Kakao, Samsung, Hyundai, Yes24, Kiwoom, SK Telecom, Toss, and Shinhan, while the global evaluated included Google, Instagram, YouTube, Netflix, Amazon, Microsoft, Apple, Paypal, Tesla, and Mastercard.

¹² Recall and related evaluation metrics are computed by comparing the result of our work against manually verified ground truth.

Table 7: Overall recall results for TLS version, cipher suite, and PQC migration requirement.

Evaluation Category	Recall (%)	Correct / Total
TLS Version	96.35	185 / 192
Cipher Suite	94.27	181 / 192
PQC Migration Requirement	98.44	189 / 192

Quantum-Readiness in Domestic and Global Enterprises Table 8 and Table 9 present the analysis results of the cryptographic algorithms used in TLS communications and the certificate status for domestic and global enterprises, respectively. According to the quantum-vulnerability and quantum-readiness analysis of this work, major domestic service providers in Korea still predominantly utilize TLS 1.2-based communication, and their certificates primarily employ the RSA algorithm, which is known to be vulnerable. In contrast, leading global enterprises have mostly adopted the latest standard, TLS 1.3, by default, and operate with cipher suites that provide a higher level of security.

In the case of Google Chrome, a phased strategy has been adopted, beginning with experiments using FN-DSA-768, and culminating in native support for quantum-resistant key exchange in Chrome versions 131 and above from 2025 onward¹³. In Korea, while the promotion of PQC-based solutions is underway, there have not yet been any instances of full-scale deployment in large, real-world services.

This analysis of major domestic and global enterprises reveals a clear contrast in quantum-readiness. While global enterprises have largely adopted the TLS 1.3 and some are even taking proactive steps by experimenting with hybrid PQC key exchange methods like ML-KEM-768_ECDHE, domestic enterprises still predominantly use TLS 1.2 and rely on vulnerable RSA-based certificates. This disparity highlights a relative lack of cryptographic agility in domestic enterprises compared to their global counterparts.

Table 8: Security level and quantum readiness in domestic enterprises (based on our system; see Section 3.4).

Enterprise	TLS ver.	Key Exchange	Cipher Suite	Certificate	Renewal Cycle	
Naver	1.3 (H)	ECDHE (L)	AES_128_SHA256 (M)	RSA (L)	90	
Coupang					393	
Kakao			AES_256_SHA384 (H)		392	
SK Telecom					396	
Samsung	1.2 (M)	ECDHE_RSA (L)	AES_128_SHA256 (M)		396	
Hyundai					322	
YES24					396	
Kiwoom			AES_256_SHA384 (H)		335	
Toss					335	
Shinhan Bank					335	

In order to provide a fair, normalized comparison between domestic and global enterprises, the security configuration data from Tables 8 and 9 are transformed into quantitative *quantum readiness* scores for five evaluation indicators. Specifically, the TLS version readiness is defined as the proportion of deployments adopting TLS 1.3, yielding 40% for domestic enterprises and 90% for global enterprises. The key exchange readiness is measured as the proportion of deployments employing medium- or high-security (M/H) post-quantum hybrid schemes, such as ML-KEM-ECDHE; the results show 0% for domestic enterprises (all low-security ECDHE) and 40% for global

¹³ See <https://blog.chromium.org/2023/08/protecting-chrome-traffic-with-hybrid.html>.

Table 9: Security level and quantum readiness in global enterprises (based on our system; see Section 3.4).

Enterprise	TLS ver.	Key Exchange	Cipher Suite	Certificate	Renewal Cycle
Google	1.3 (H)	ML-KEM-768_ECDHE (H)	AES_128_SHA256 (M)	ECC (L)	84
Instagram					90
YouTube					84
Netflix		ECDHE (L)		RSA (L)	396
Amazon					359
Microsoft					180
Apple					85
PayPal					364
Tesla	ECDHE (L)	AES_256_SHA384 (H)	90		
Mastercard	1.2 (M)		ECDHE_RSA (L)	364	

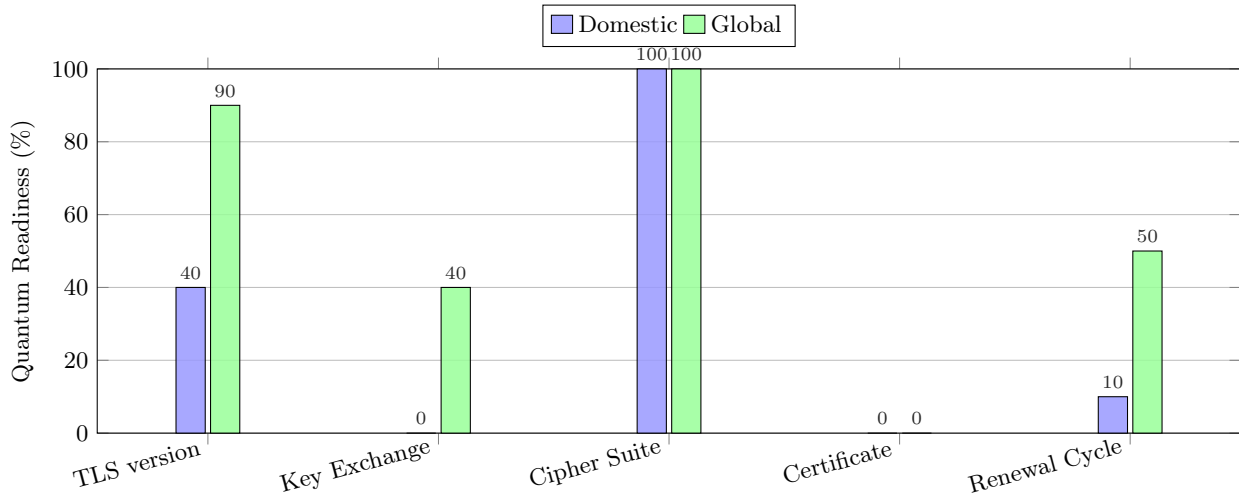


Fig. 7: Comparison of quantum readiness across five indicators: TLS version, key exchange, cipher suite, certificate, and renewal cycle.

enterprises. For the cipher suite readiness, the proportion of deployments using M/H-level cipher suites (with no low-security suites) is computed, resulting in 100% for both groups. The certificate readiness is defined as the proportion of certificates based on ECC or post-quantum algorithms; in this experiment, no such certificates are observed for either domestic or global enterprises (0%). Finally, the renewal cycle (expressed in days) is determined as the proportion of certificates with a validity period of 90 days or less, with domestic enterprises at 40% and global enterprises at 50%. In all five indicators, a higher percentage reflects a greater degree of preparedness for post-quantum security migration (see Figure 7).

Certificate Expiry and Migration Based on our analysis of certificate validity periods and associated vulnerability levels, it is possible to determine an appropriate timeline for timely PQC migration. In this case study, it is observed that many certificates are issued with start and expiration dates that are approaching soon. While there is a global shift toward drastically reducing certificate renewal periods to minimize exposure time¹⁴, domestic

¹⁴ Internationally, there is a growing mandate to gradually reduce the maximum validity period of certificates to as little as 47 days. See <https://www.digicert.com/blog/tls-certificate-lifetimes-will-officially-reduce-to-47-days>.

enterprises generally continue to use certificates with validity periods close to one year¹⁵. This indicates that domestic enterprises have a less agile certificate management policy, which could hinder a rapid transition to PQC. The longer validity periods mean that even if a new, quantum-safe certificate is available, it might take a significant amount of time for all systems to be updated, as they would have to wait for existing certificates to expire.

Toward Quantum-Readiness Infrastructure The overall analysis shows that the quantum security preparedness of major domestic enterprises is notably lower than that of their global counterparts. This is due to a clear tendency for domestic enterprises to rely on TLS 1.2 and vulnerable cryptographic algorithms. As the threat of quantum computing becomes a reality, this cryptographic asymmetry could lead to significant long-term security risks.

Therefore, domestic enterprises must urgently establish a PQC transition roadmap. This strategy should be comprehensive, moving beyond simple TLS version upgrades to include the replacement of conventional RSA algorithms with key exchange and authentication mechanisms based on PQC. The analytical framework proposed in this study is expected to play a crucial role in helping enterprises objectively diagnose their current quantum vulnerabilities and develop effective migration plans.

5 Conclusion

This paper presented a framework for automatically detecting the cryptographic algorithms used in the TLS protocol and identifying those vulnerable in the presence of quantum computers. By analyzing network packets to extract cipher suite information and applying targeted filtering strategies, including a method for examining encrypted TLS 1.3 certificates without decryption, the framework enables systematic evaluation of TLS deployments with respect to quantum-related vulnerabilities. These results can serve as a practical basis for assessing the necessity of migrating to post-quantum cryptographic algorithms.

Our approach achieves over 96% accuracy in packet-based detection. Notably, the proposed hybrid certificate parsing strategy minimizes unnecessary decryption overhead by combining lightweight OID-based pattern matching with selective OpenSSL parsing. This design not only preserves parsing accuracy but also reduces processing time in large-scale analyses, making the framework more practical for real-time TLS traffic evaluation.

Furthermore, we collected TLS packet data from both domestic and international organizations to assess their quantum readiness using our framework. The results indicate that global businesses exhibit higher quantum-readiness, with a TLS 1.3 adoption rate of 90%, a PQC hybrid key exchange adoption rate of 40%, and certificate with a short cycle (i.e., ≤ 90 days) ratio of 50%. In contrast, domestic deployments show a TLS 1.3 adoption rate of 40%, a PQC hybrid key exchange adoption rate of 0%, and a short-lived certificate ratio of 10%. These findings highlight the urgent need for domestic services to upgrade TLS versions and transition to PQC-based cryptography.

In our future work, we will conduct benchmarking against commercial portfolios such as Palo Alto, PQStation, and SandboxAQ, with the goal of situating our open-source framework within the broader ecosystem of enterprise-grade PQC migration solutions. Although the current implementation is limited to network protocols, future work will extend the framework to detect vulnerable cryptographic components within internal systems. We also aim to explore AI-driven automated packet analysis in more advanced scenarios, such as the examination of encrypted packets in QUIC, thereby broadening the applicability and impact of our approach.

6 Acknowledgment

This work was supported by Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government(MSIT) (No.RS-2025-02306395, Development and Demonstration of PQC-Based Joint Certificate PKI Technology, 50%) and this work was supported by the Institute of Information & Communications Technology Planning & Evaluation(IITP) grant funded by the Korea government(MSIT) (No. RS-2025-25394739, Development of Security Enhancement Technology for Industrial Control Systems Based on S/HBOM Supply Chain Protection, 50%).

¹⁵ With the exception of Naver, the domestic enterprises survey all had certificate validity periods of approximately one year, whereas about half of the global enterprises had periods of around 90 days.

References

1. E. Rescorla, “The Transport Layer Security (TLS) Protocol Version 1.3,” Request for Comments RFC 8446, Internet Engineering Task Force, 2018.
2. P. W. Shor, “Algorithms for quantum computation: discrete logarithms and factoring,” in *Proceedings 35th annual symposium on foundations of computer science*, pp. 124–134, Ieee, 1994.
3. R. L. Rivest, A. Shamir, and L. Adleman, “A method for obtaining digital signatures and public-key cryptosystems,” *Communications of the ACM*, vol. 21, no. 2, pp. 120–126, 1978.
4. N. Koblitz, A. Menezes, and S. Vanstone, “The state of elliptic curve cryptography,” *Designs, codes and cryptography*, vol. 19, pp. 173–193, 2000.
5. T. Dierks and E. Rescorla, “The Transport Layer Security (TLS) Protocol Version 1.2,” Request for Comments RFC 5246, Internet Engineering Task Force, 2008.
6. D. Cooper, S. Santesson, S. Farrell, S. Boeyen, R. Housley, and W. Polk, “Internet x. 509 public key infrastructure certificate and certificate revocation list (crl) profile,” tech. rep., 2008.
7. M. Ounsworth, J. Gray, M. Pala, and J. Klaußner, “Composite public and private keys for use in internet pki,” 2023.
8. C. Wang, W. Xue, and J. Wang, “Integration of quantum-safe algorithms into x. 509v3 certificates,” in *2023 IEEE 3rd International Conference on Electronic Technology, Communication and Information (ICETCI)*, pp. 384–388, IEEE, 2023.
9. S. Paul, Y. Kuzovkova, N. Lahr, and R. Niederhagen, “Mixed certificate chains for the transition to post-quantum authentication in tls 1.3,” in *Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security*, pp. 727–740, 2022.
10. A. C. Chen, “Post-quantum cryptography x. 509 certificate,” in *2024 International Conference on Smart Systems for applications in Electrical Sciences (ICSSES)*, pp. 1–6, IEEE, 2024.
11. L. Chen, S. Jordan, Y.-K. Liu, D. Moody, R. Peralta, R. Perner, and D. Smith-Tone, “NIST released NISTIR 8105 report on post-quantum cryptography,” *Nat. Inst. Standards Technol., Gaithersburg, MD, USA, Tech. Rep.*, 2016.
12. P.-A. Fouque, J. Hoffstein, P. Kirchner, V. Lyubashevsky, T. Pornin, T. Prest, T. Ricosset, G. Seiler, W. Whyte, Z. Zhang, *et al.*, “Falcon: Fast-fourier lattice-based compact signatures over NTRU,” *Submission to the NIST’s post-quantum cryptography standardization process*, vol. 36, no. 5, pp. 1–75, 2018.
13. C. A. Melchor, N. Aragon, S. Bettaleb, L. Bidoux, O. Blazy, J.-C. Deneuville, P. Gaborit, E. Persichetti, G. Zémor, and I. Bourges, “Hamming quasi-cyclic (HQC),” *NIST PQC Round*, vol. 2, no. 4, p. 13, 2018.
14. National Institute of Standards and Technology, “Status report on the third round of the nist post-quantum cryptography standardization process.” <https://nvlpubs.nist.gov/nistpubs/ir/2024/NIST.IR.8547.ipd.pdf>, 2024. NIST IR 8547 (Initial Public Draft), accessed on 2025-07-21.
15. L. K. Grover, “A fast quantum mechanical algorithm for database search,” in *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*, pp. 212–219, 1996.
16. N. J. AlFardan and K. G. Paterson, “Lucky Thirteen: Breaking the TLS and DTLS Record Protocols,” in *Proceedings of the 2013 IEEE Symposium on Security and Privacy*, Springer, 2013.
17. B. Möller, T. Duong, and K. Kotowicz, “This POODLE Bites: Exploiting The SSL 3.0 Fallback,” tech. rep., Google, 2014.
18. S. Jaques, M. Naehrig, M. Roetteler, and F. Virdia, “Implementing Grover oracles for quantum key search on AES and LowMC,” Cryptology ePrint Archive, Report 2019/1146, 2019. <https://eprint.iacr.org/2019/1146>.
19. K. Jang, A. Baksi, H. Kim, G. Song, H. Seo, and A. Chattopadhyay, “Quantum analysis of AES,” *IACR Communications in Cryptology*, vol. 2, no. 1, 2025.
20. H. Shi and X. Feng, “Quantum circuits of aes with a low-depth linear layer and a new structure,” *Cryptology ePrint Archive*, 2024.
21. W. Castryck and T. Decru, “An efficient key recovery attack on sidh,” in *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pp. 423–447, Springer, 2023.
22. K. Scarfone, T. Polk, L. Ouyang, and M. Davidson, “Considerations for achieving cryptographic agility: Strategies and practices,” Tech. Rep. CSWP 39 (Initial Public Draft), National Institute of Standards and Technology (NIST), March 2025. Initial Public Draft.
23. IBM, “Ibm guardium for quantum-safe data security.” <https://www.ibm.com/products/guardium-quantum-safe>, 2024. Accessed: 2025-07-21.
24. TYCHON, “Pqc management module.” <https://tychon.io/products/tychon/pqc-management-module/>, 2024. Accessed: 2025-07-21.
25. A-Packets, “A-packets: Ai-powered network security solutions.” <https://apackets.com/>, 2024. Accessed: 2025-07-21.
26. Palo Alto Networks, “Post-quantum migration planning and preparation.” <https://docs.paloaltonetworks.com/network-security/quantum-security/administration/quantum-security-concepts/post-quantum-migration-planning-and-preparation>, 2024. Accessed: 2025-08-23.

27. PQStation, “Qvision: Cryptographic visibility and pqc migration support.” <https://www.pqstation.com/offerings/qvision>, 2025. Accessed: 2025-08-23.
28. SandboxAQ, “Aqtive guard: Unified cryptography management.” <https://www.sandboxaq.com/solutions/aqtive-guard>, 2025. Accessed: 2025-08-23.
29. C. L. Hedrick, “A Standard for the Transmission of IP Datagrams over Ethernet Networks,” Request for Comments RFC 894, Internet Engineering Task Force, 1984. Ethernet.
30. S. T. Valapu and J. Heidemann, “Towards a Non-Binary View of IPv6 Adoption,” *arXiv preprint*, 2025. arXiv:2507.11678.
31. E. Rescorla, “HTTP Over TLS,” Request for Comments RFC 2818, Internet Engineering Task Force, 2000. Informational.
32. T. Dierks and C. Allen, “The Transport Layer Security (TLS) Protocol Version 1.0,” Request for Comments RFC 2246, Internet Engineering Task Force, 1999.
33. J. Reynolds and J. Postel, “Assigned Numbers,” Request for Comments RFC 1700, Internet Engineering Task Force, October 1994. Obsoleted by: RFC 3232.
34. “Service name and transport protocol port number registry,” 2025. Accessed: 2025-07-22.
35. The Apache Software Foundation, “Apache tomcat 9 (9.0.107) - ssl/tls configuration how-to,” 2025. Accessed: 2025-07-22.
36. D. Eastlake, “Transport Layer Security (TLS) Extensions: Extension Definitions,” Request for Comments RFC 6066, Internet Engineering Task Force, 2011.
37. A. Ghedini and M. Varga, “(SNI) Encryption,” Request for Comments RFC 8744, Internet Engineering Task Force, 2020.
38. K. Moriarty and S. Farrell, “Deprecating TLS 1.0 and TLS 1.1,” Request for Comments RFC 8996, Internet Engineering Task Force, 2021.
39. D. Cooper, S. Santesson, S. Farrell, R. Boeyen, W. Housley, and T. Polk, “Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile,” Request for Comments RFC 5280, Internet Engineering Task Force, 2008.
40. E. de la Hoz, G. Cochrane, J. M. Moreira-Lemus, R. Paez-Reyes, I. Marsa-Maestre, and B. Alarcos, “Detecting and defeating advanced man-in-the-middle attacks against tls,” in *2014 6th International Conference on Cyber Conflict (CyCon)*, pp. 210–221, NATO CCD COE Publications, Tallinn, 2014.
41. Federal Office for Information Security (BSI), “Technical Guideline TR-02102-2: Cryptographic Mechanisms - Recommendations and Key Lengths, Part 2 – Use of Transport Layer Security (TLS),” tech. rep., Federal Office for Information Security (BSI), Jan. 2025. Accessed: 2025-07-29.
42. National Institute of Standards and Technology, “Module-lattice-based digital signature standard,” Federal Information Processing Standard FIPS 204, U.S. Department of Commerce. Final version.
43. S. Santesson, W. Polk, P. Barzin, and M. Nystrom, “Internet X.509 Public Key Infrastructure Qualified Certificates Profile,” Request for Comments RFC 3039, Internet Engineering Task Force, 2001.
44. The OpenSSL Project Authors, *ASN1_TIME_set and Related Functions - OpenSSL Manual*. OpenSSL Project, 2025. Accessed: 2025-07-29.
45. Wireshark Foundation, “Wireshark - Official Website.” <https://www.wireshark.org/>, 2025. Accessed 2025-07-27.
46. Wireshark Foundation, “Transport Layer Security (TLS) - Wireshark Wiki.” <https://wiki.wireshark.org/TLS>, 2020. Accessed 2025-07-27.